

Idea: Data on the range for each batch will improve training

Typically:

$$y = \sigma(Wx + b)$$

Apply batch normalization prior to activation

$$r = Wx + b$$

$$r^{BN} = \frac{r - \mu_r}{\sigma_r} \odot \alpha + \beta$$

r - aggregations for a batch of data

$\vec{r}$ - each instance in batch is a vector

$\bar{\mu}_r$ - mean of each feature

$\bar{\sigma}_r$ - standard deviation for each feature

α - parameter that scales distribution

β - parameter that shifts distribution

$\kappa(-BN)$

μ

$$\gamma = \sigma(r^{BN})$$

α, β are learnable parameters

- do backprop to compute $\frac{\partial L}{\partial \alpha}, \frac{\partial L}{\partial \beta}$

update with gradient descent.

At inference time:

Compute $\mu_{\text{Train}}, \sigma_{\text{Train}}$

Use these values

This forces inputs to subsequent layers to have same range of values

Easier to generalize across distributions with different shifts/scales.

Randomness of batching adds noise to shifting/scaling $\Rightarrow$ reduces overfitting.

Autoencoders

Encoder: $h = f(x)$

Decoder: $\hat{x} = g(h)$

Goal: Reconstruct the output $\hat{x}$ to be as close as possible to input x .

Minimize $L(x, g(f(x)))$

Typically $L = \frac{1}{2} \sum_i (x_i - \hat{x}_i)^2$

Undercomplete autoencoders:

$$\text{size}(h) < \text{size}(x)$$

Overcomplete autoencoders:

$$\text{size}(h) > \text{size}(x)$$

h is low-dimensional representation of x

Example

Example

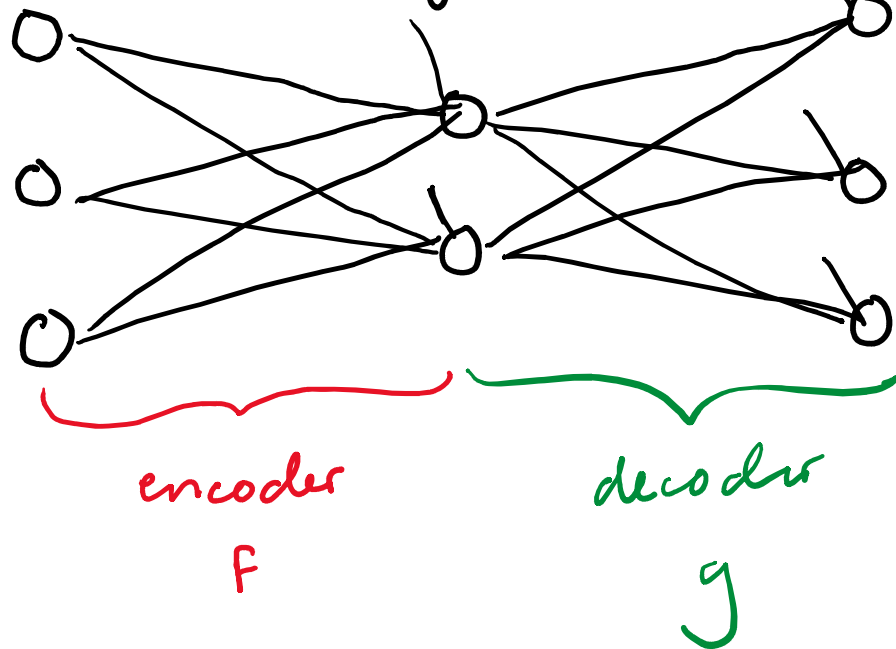
3. element input x

2. element representation h

Input layer

Hidden layer

Output layer



Denoising Autoencoder

Can remove noise from inputs

Noise model: $\tilde{x} = n(x)$

Encoder: $h = f(\tilde{x})$

Decoder: $\hat{x} = g(h)$

Compute loss as:

$$\frac{1}{n} \sum_{i=1}^n ||x_i - \hat{x}_i||^2$$

Compute loss as:

$$L(x, g(f(\tilde{x})))$$

Learn encodings that are robust to noise.